Phoenix V02

Agentic Output Summary

A consolidated record of the porting and feature-development passes executed against the V12 PhoenixSketch codebase. Each chapter summarizes one work unit -- what was done, the engineering rationale, the resulting end-state, and the deferred follow-ups. Sources for each chapter are collected in the Sources appendix at the end. The final addendum reproduces the T-41 menu hierarchy as a navigation reference.

Contents

1. Architecture Survey: Phoenix vs T-41
2. NFM Demod Port
3. CW Message Keyer Port
4. USB Input Bundle Port (Mouse + Keyboard + USB Host)
5. WSJT-X CAT Comparison and Compatibility Additions (TB, SF)
6. PSK31 Primitives Port
7. FT8 Port: Enum, Dispatcher, Vendored ft8_lib, Phoenix Integration
8. FT8 Message Pane (Three-Column Display)
9. Loop Wire-In and DEMODULATION Button Cycle Extension
10. FT8-Tuned Receive Filter, Callsign/Grid Persistence, and FT8 Transmit Wiring
11. CAT Recognition for FT8 and NFM Modes
12. Fine-Tune Encoder Repurposed for FT8 RX Frequency
13. FT8 Status Line on the Pane
14. Slot-Aligned TX Gate (Per-Slot, Even/Odd Interval)
15. Polyphase Upsampling 12 to 24 kHz
16. BandEQ Bypass During FT8 TX
17. UTC Sub-Second TX Alignment
18. FT8 Menu Page (Variables + Queue Presets)
19. CL/GR CAT Commands (Callsign/Grid)
20. Per-Band FT8 Frequency Presets + Tune Menu Entry
21. One-Click "Go to FT8 + Tune"
22. USB-Keyboard Text Editor (Soft Modal)
23. Target-Callsign Tracking from RX
24. USBHost_t36 Vendoring and Repointing

Sources

Addendum: T-41 Menu Hierarchy

1. Architecture Survey: Phoenix vs T-41

Two divergent forks of the same hardware (T41-EP V12 transceiver). PhoenixSketch is a ground-up rewrite organized around hardware boards (BPFBoard, LPFBoard, RFBoard, MainBoard_*, FrontPanel_*) with explicit state machines (ModeSm, UISm, PowerCalSm, ReceiveIQCalSm, TransmitCarrierCalSm, TransmitIQCalSm, all generated by StateSmith from .drawio diagrams). T41_SDR is a feature-organized fork by tmr4 with additional capabilities Phoenix lacks: FT8, PSK31, WSJT, mouse, keyboard, USB host, beacon monitor, remote display. The two SDT.h files differ substantially: include guards, state/mode constants, demod modes.

What was done

- PhoenixSketch baseline: 87 files, ~2.3 MB, board-organized layout.
- T41_SDR baseline: ~80 files, board+feature mix; KGOBA's ft8_lib vendored in a src/ subtree.
- Divergence is fundamental, not stylistic: separate enumerations, separate audio pipelines, separate state-management approaches.

Engineering rationale

Naive file-by-file diff would silently break things. Plan locked in: feature-by-feature port with explicit scope. Each feature gets its own pass with backup-before-edit, plan-then-execute, verify-with-diff.

End state

Backup snapshot taken at code/src/PhoenixSketch.bak/ before any edits. Per-feature port queue established.

Caveats / deferred work

T41 features that have no Phoenix equivalent (display panes, FT8 UI, QSO automation) require Phoenix-native designs rather than literal copies. The vendored upstream library (ft8_lib) is the same in both.

2. NFM Demod Port

First feature port. Phoenix had no narrow-band FM demodulation. T41 has a single small function nfmdemod() in Demod.cpp implementing Lyons (Understanding DSP, sec 13.22) quadrature FM phase-derivative demodulation.

What was done

- Added enum value NFM = 5 to ModulationType in SDT.h (slotted before DCF77 = 29 so persisted JSON values don't shift).
- Added nfmdemod(DataBlock*) function to DSP.cpp.
- Adapted from T41's interleaved-I/Q layout to Phoenix's separate data->I[] / data->Q[] arrays.
- Added case NFM in Demodulate() dispatcher in DSP.cpp.
- Declared nfmdemod() public in DSP.h.

Engineering rationale

Smallest-scope port to validate the workflow. Single function, self-contained DSP, no UI surface required. Used a local prev_i shadow to handle the in-place I-overwrite correctly: T41's algorithm reads input[i-1] but Phoenix overwrites data->I[i] with audio in place, so we must shadow the previous raw I in a local before writing audio out.

End state

Selecting modulation NFM routes audio through the new demod path. Compiles, links, runs identically to before until NFM is selected. Brace balance preserved on all three touched files.

Caveats / deferred work

De-emphasis filter intentionally not ported (T41 had it commented out). Receive filter chain is still SSB-shaped; NFM through SSB filters is audible but not optimal. Display label and CAT recognition deferred.

3. CW Message Keyer Port

T41 has an automated CW message keyer (keyer.cpp, ~561 lines): pre-stored messages, character-to-Morse encoding tables, dit/dah timing, raised-cosine envelope shaping. Triggered by USB keyboard. Phoenix has DSP_CWProcessing.cpp for receive-side CW decoding only — no transmit-side message keyer.

What was done

- Architectural mismatch surfaced before any edits: T41 is blocking, polled, USB-keyboard-driven; Phoenix is non-blocking, FSM-driven (CW transmit states ModeSm: CW_TRANSMIT_DIT_MARK, CW_TRANSMIT_KEYER_SPACE, CW_TRANSMIT_KEYER_WAIT).
- Plumbing-only port chosen: data tables + storage + KeyerSetup() working; SendMessage() and KeyerLoop() stubbed with concrete TODOs.
- New file DSP_CWMessageKeyer.h (89 lines): storage globals, size limits, function declarations.
- New file DSP_CWMessageKeyer.cpp (202 lines): Morse encoding tables (letter, number, parallel punctuationASCII/punctuationCode), KeyerSetup() builds raised-cosine ramps, MorseCodeForChar() works, sender/loop stubs.
- Default messages use neutral placeholders ("CQ CQ CQ DE <CALL>", "73", "599") rather than T41's hardcoded KN6ZDE callsign.

Engineering rationale

A faithful port of T41's blocking SendMessage() would freeze Phoenix's main loop during transmission, breaking the radio's responsiveness. Plumbing-only sets up the data foundation so once the USB keyboard ports next, the keyer becomes wirable; the actual sending engine is a follow-up that drives Phoenix's existing FSM by injecting synthetic iDIT_PRESSED / iDAH_PRESSED interrupts.

End state

Compiles, links, dormant. MorseCodeForChar() and the encoding tables are usable today. Brace balance: 17/17 cpp, 0/0 header. Zero symbol collisions with existing Phoenix code.

Caveats / deferred work

Non-blocking message-send engine deferred (concrete design noted in TODO comments). Persistence of edited messages across power cycles deferred. T41's USB keyboard input dispatch deferred (depends on USB keyboard port).

4. USB Input Bundle Port (Mouse + Keyboard + USB Host)

T41 has three coordinated files for USB host input: t41USBHost.cpp (driver), keyboard.cpp (HID keyboard), mouse.cpp (HID mouse). All gated by HOST_KEYBOARD_MOUSE_SUPPORT compile flag.

What was done

- Three new files in Phoenix matching FrontPanel_* convention: FrontPanel_USBHost.h/.cpp (~84 lines), FrontPanel_USBKeyboard.h/.cpp (~142 lines), FrontPanel_USBMouse.h/.cpp (~124 lines).
- New Config.h flag USB_HOST_INPUT_ENABLED (commented out by default).
- Faithful keyboard port including the caps-lock+space/backspace bug workaround for PJRC's library.
- Critical rename: T41's bare getc()/putc() renamed to KeyboardReadChar()/KeyboardWriteChar() to avoid clashing with stdio.h's getc/putc (Phoenix's SDT.h includes <stdio.h>).
- Mouse driver layer only: MouseSetup, MouseLoop, MousePollEvent. Action dispatch (clicks-to-pane-behavior) intentionally not ported.

Engineering rationale

T41's mouse.cpp depends on ~15 T41-specific globals (t41.DemodMode, centerLine, FT8MsgWindowClick, MenuBarSelect, etc.) that Phoenix doesn't have. The right place for clicks-in-spectrum-pane semantics is Phoenix's UI layer, not a literal copy of T41's mouse action dispatch. Keyboard ring buffer and USB host driver are clean and portable; mouse driver layer (raw events) is portable; mouse action layer needs a Phoenix-native rewrite.

End state

With flag off (default): all three files compile to near-empty, no library dependency, zero runtime impact. With flag on: an attached USB keyboard fills kbBuffer[], drainable via KeyboardReadChar(); attached mouse fills MouseEvent snapshots drainable via MousePollEvent().

Caveats / deferred work

Wiring InitializeUSBHost() into setup() and TickUSBHost() into loop() deferred (one-line each, intentionally separate pass). Mouse action dispatch deferred (Phoenix-native UI binding). USB serial pass-through and CAT-over-USB-host (T41 features) not in scope.

5. WSJT-X CAT Comparison and Compatibility Additions (TB, SF)

Discovery: T41's wsjt.cpp is not a digital-mode decoder — it's a CAT control adapter for the WSJT-X desktop application, listening on a second USB serial port and emulating Kenwood TS-890S/TS-2000 commands.

What was done

- Phoenix's existing CAT.cpp emulates Kenwood TS-480 on SerialUSB1 (25 commands).
- Diffed Phoenix's CAT against the WSJT-X command list. WSJT-X needs: ID, PS, IF, AI, KS, FA, FT, MD, TX, RX, TB, SF.
- All but TB and SF were already implemented in Phoenix's CAT.cpp.
- Added TB_read/TB_write: returns TB0; (no split mode), accepts writes silently.
- Added SF_read/SF_write: SFx; returns SFx<11-digit-freq><mode-digit>;.
- Bumped NUM_SUPPORTED_COMMANDS from 25 to 27.
- WSJT-X usage path: configure WSJT-X with rig profile = Kenwood TS-480.

Engineering rationale

Vendoring T41's separate WSJT-X CAT module would be redundant — Phoenix already has 85% of the WSJT-X command set via the existing CAT. Two small handlers close the gap with zero new infrastructure.

End state

WSJT-X with TS-480 profile can now control Phoenix: frequency, mode, power, PTT, status all work via the existing CAT channel.

Caveats / deferred work

First WSJT-X connection may surface formatting nits in IF; field widths. MD; with FT8 mode added in a separate later pass.

6. PSK31 Primitives Port

T41's psk31.cpp (995 lines) contains the standard G3PLX varicode tables, varicode encoder/decoder, a clean DBPSK demodulator, plus five experimental phase-shift detectors with hand-tuned magic constants and no symbol-clock synchronization.

What was done

- Honest assessment surfaced: T41's PSK31 algorithm is experimental, debug-spam-laden, and lacks symbol-clock recovery. The reusable parts are the varicode tables and the DBPSK function.
- Plumbing-only port: data tables + clean DBPSK demod, no decoder zoo.
- New file DSP_PSK31.h (91 lines): public API.
- New file DSP_PSK31.cpp (216 lines): G3PLX 128-entry varicode table, 64-entry bitmask helper, varicode encoder/decoder (no Serial.print spam), DBPSK decoder adapted to Phoenix's separate I/Q DataBlock.
- Exposed Phoenix's existing ApproxAtan2 in DSP.h for the DBPSK consumer.

Engineering rationale

Five competing detector implementations with hand-tuned thresholds (PSK_MULT, PSK_THRESHOLD, PSK_TIME, PSK_WIGGLE) and Serial.print on every input bit don't belong in production. A proper Phoenix PSK31 implementation should be written from scratch using Phoenix's FFT/Goertzel infrastructure.

End state

Phoenix can now translate ASCII <-> PSK31 varicode and do DBPSK symbol decoding on complex baseband samples. Primitives are dormant until wired up. Spot-checks pass: space=0b1, 'e'=0b11, LF=0b11101, CR=0b11111.

Caveats / deferred work

Symbol-clock recovery (Gardner / early-late / Mueller-Muller) deferred. Audio bandpass filter for PSK31 deferred. UI / decoded-text display surface deferred. T41's experimental detectors and WAV-file test path intentionally not ported.

7. FT8 Port: Enum, Dispatcher, Vendored ft8_lib, Phoenix Integration

T41's FT8 implementation is two layers: ft8.cpp (1814 lines integration) and src/ft8_lib/ (5,194 lines vendored upstream KGOBA library). Phoenix has neither. Largest single port in the work.

What was done

- Vendored ft8_lib (24 files, 5,194 lines) into code/src/PhoenixSketch/src/ft8_lib/ verbatim. KISS FFT, LDPC, CRC, FT8 message encode/decode, real-time monitor.
- Added FT8_INTERNAL = 6 to ModulationType enum in SDT.h (slotted between NFM = 5 and DCF77 = 29).
- Added case FT8_INTERNAL to Demodulate() dispatcher calling new ft8InternalDemod(DataBlock*).
- Declared ft8InternalDemod() in DSP.h.
- New file DSP_FT8.h (160 lines): public API mirrored from T41's ft8.h (RxMsg, TxMsg, QsoView structs; window/state constants; Initialize, Buffer, RunDecoderLoop, change handlers).
- New file DSP_FT8.cpp (initially 486 lines): Phoenix-side integration. AddDecodedMessage callback that ft8_lib calls back into; BufferFT8Data accumulates 12 kHz audio into a 720 KB EXTMEM ft8SlotBuf; RunFT8DecoderLoop FSM (BUFFERING -> PROCESSING -> DECODING -> RX_UPDATE -> TX -> TX_UPDATE); ft8InternalDemod decimates from Phoenix's I/Q sample rate to 12 kHz via integer decimation.

Engineering rationale

ft8_lib is the upstream signal-processing engine; vendoring it pins the version. Phoenix's integration layer must provide AddDecodedMessage (callback target for ft8_lib decoder hits) and the audio-buffering pipeline. T41-specific UI dependencies (display globals, FT8 message windows, QSO automation triggers) intentionally stubbed for first cut.

End state

Selecting FT8_INTERNAL activates the FT8 path. ft8_lib fully built and linked: ft8_encode, ftx_decode_candidate, ldpc_decode, KISS FFT all callable. Total Phoenix tree growth: +6,977 lines.

Caveats / deferred work

Decoded messages have nowhere to render yet (deferred to FT8 pane pass). TX modulator stubbed (deferred). Audio bandpass not FT8-tuned (deferred). Callsign defaults to MY_CALL ("ABCDE") and grid "AA00" (deferred). RunFT8DecoderLoop not yet wired into Loop.cpp (deferred).

8. FT8 Message Pane (Three-Column Display)

Decoded FT8 messages had nowhere to render. T41 has three FT8 sub-windows (All / CQ / RX). Phoenix has no FT8 panes.

What was done

- T41-style three-column layout chosen.
- New file MainBoard_DisplayFT8.h (35 lines): pane API.
- New file MainBoard_DisplayFT8.cpp (138 lines): renders three columns (All / CQ / RX) inside a given rect, each with header + 2-line per-message rows (timestamp/freq/SNR on row 1, message body on row 2).
- Modified DSP_FT8.h to expose rxBuf/decodedMsgs/allList/cqList/rxList (removed static qualifiers in DSP_FT8.cpp).
- Modified DSP_FT8.cpp::DisplayAllMessages stub to set PaneSpectrum.stale = true on each decode cycle.
- Modified MainBoard_DisplayHome.cpp::DrawSpectrumPane to dispatch to DrawFT8MessageList when modulation == FT8_INTERNAL.
- Added (NFM) and (FT8) modulation labels to DrawFreqBandModPane's switch.

Engineering rationale

Reusing the spectrum pane area (520x345 px) is the only place big enough for three FT8 columns. Conditional dispatch in DrawSpectrumPane keeps the pane infrastructure intact; the existing stale-flag mechanism handles redraw timing. Mode-conditional rendering follows the existing PaneSAMOffset pattern (only renders when modulation is SAM).

End state

When ED.modulation == FT8_INTERNAL, the spectrum pane swaps to the three-column FT8 message list. AddDecodedMessage updates rxBuf and the lists; DisplayAllMessages flags stale; DrawDisplay repaints. ~9 messages visible per column at default font.

Caveats / deferred work

RX column filter uses +/- 50 Hz against ft8RxFreq (heuristic). Pane is read-only — clicks/scrolls not wired (mouse driver layer exists; binding deferred). No on-screen display of ft8RxFreq yet (deferred to status-line pass).

9. Loop Wire-In and DEMODULATION Button Cycle Extension

FT8 subsystem was implemented but not yet driven from the main loop. Modulation cycling via the front-panel DEMODULATION button stopped at SAM, never reaching NFM or FT8_INTERNAL.

What was done

- Added InitializeFT8() call to setup() in Globals.cpp after InitializeSignalProcessing() and before InitializeAudio().
- Added RunFT8DecoderLoop() call to loop() in Loop.cpp after PerformSignalProcessing() and before DrawDisplay() (Step 2.5).
- Extended the DEMODULATION button cycle in two places (HOME/UPDATE state at line 478 and CALIBRATE_FREQUENCY state at line 673) using an explicit cycle[] array {USB, LSB, AM, SAM, NFM, FT8_INTERNAL}.
- Added DSP_FT8.h include to Globals.cpp and Loop.cpp.

Engineering rationale

Explicit cycle array is clearer than the original ++/wraparound logic and makes the included modes obvious. IQ and DCF77 are intentionally skipped (not normal RX modes). RunFT8DecoderLoop is a near-zero-cost no-op when FT8 isn't initialized or when no samples are flowing in.

End state

Press DEMODULATION on the front panel: cycles USB -> LSB -> AM -> SAM -> NFM -> FT8_INTERNAL -> USB. Modulation label updates accordingly. FT8 mode now activates the spectrum pane swap.

Caveats / deferred work

FIR filter not yet FT8-tuned (deferred). Slot-aligned TX gating not yet (deferred).

10. FT8-Tuned Receive Filter, Callsign/Grid Persistence, and FT8 Transmit Wiring

Three coordinated follow-ups in one pass: tune the receive filter for FT8 audio bandwidth, persist operator identity, and wire transmit-side audio into Phoenix's TX pipeline.

What was done

- Filter (DSP_FIR.cpp +13 lines): added case NFM and case FT8_INTERNAL to InitFilterMask switch (previously fell through silently). NFM gets +/- 3 kHz; FT8 gets 200-3000 Hz (wide enough for ft8_lib's STFT to do its own fine selection).
- Persistence (SDT.h + Storage.cpp + DSP_FT8.cpp +20 lines): added ED.callsign[14] = MY_CALL and ED.grid[7] = "AA00" fields; added JSON serialization in SaveDataToStorage and deserialization in both RestoreDataFromStorage and RestoreDataFromSDCard; updated InitializeFT8 to read from ED.
- Transmit (DSP_FT8.h + DSP_FT8.cpp + DSP.cpp +105 lines): four new public functions FT8QueueAndStartTx / FT8IsTxInProgress / FT8GetNextTxAudioChunk / FT8CancelTx; TX state vars; nearest-neighbor 2x upsample 12->24 kHz (replaced in a later pass); RunFT8DecoderLoop TX state actually triggers; PTT release-on-completion falling-edge detector; DSP.cpp::TransmitProcessing substitutes FT8 audio for mic audio when FT8 TX is in progress.

Engineering rationale

Filter case was a pre-existing bug (uninitialized low_Hz/high_Hz when those modes were selected). Persistence avoids hardcoding callsign in Config.h. Transmit is the larger half-day of work — the audio-substitution point in TransmitProcessing was chosen after decimation (24 kHz / 256 samples) so the rest of the SSB pipeline (Hilbert, sideband selection, interpolation) handles modulating the FT8 audio onto IQ output the same way it handles mic audio.

End state

FT8 audio: 200-3000 Hz passband. Operator identity persists across power cycles. Queue a message into txBuf[0] WAITING, set ft8TxState=1, and the next decode cycle's TX state fires FT8QueueAndStartTx + PTT_PRESSED; PTT_RELEASED dispatches automatically when audio drains.

Caveats / deferred work

Even/odd interval gating intentionally ignored (deferred). Nearest-neighbor upsampling generates spectral images (replaced with polyphase later). BandEQ runs over FT8 audio (bypassed later). Tx timing sub-second precision deferred. No UI to populate txBuf yet (deferred to FT8 menu page).

11. CAT Recognition for FT8 and NFM Modes

MD; CAT command returned ?; for FT8_INTERNAL and NFM. WSJT-X-style remote rig control needed Kenwood-format mode reporting.

What was done

- Added Kenwood TS-480 mode mappings: 4 = FM (Phoenix NFM), 9 = DATA (Phoenix FT8_INTERNAL).
- MD_read: returns MD4; for NFM, MD9; for FT8_INTERNAL.
- MD_write: case 4 sets bands[..].mode = NFM and ED.modulation = NFM; case 9 sets both to FT8_INTERNAL. Both fire iMODE.
- IF_read inner mode switch: NFM -> 4, FT8_INTERNAL -> 9.
- SF_read inner mode switch: same mappings.
- ED.modulation is mirrored explicitly in the new MD_write cases (the existing LSB/USB/AM/SAM cases rely on a band-change resync that doesn't fire for direct CAT mode-changes).

Engineering rationale

MD9 is the most accurate Kenwood mapping for FT8 / data mode. WSJT-X configured for a TS-480 will recognize MD9 as DATA in its UI. The explicit ED.modulation mirror makes the new cases work correctly even though Phoenix's existing cases have a different sync pattern.

End state

CAT clients (WSJT-X, hand-typed terminal) now see FT8 as MD9 / NFM as MD4. MD9; from a CAT host switches Phoenix to FT8_INTERNAL immediately. Status string IF; reports the right mode digit.

Caveats / deferred work

Some WSJT-X profiles for older Kenwood rigs expect MD2 (USB) for FT8 instead of MD9 (DATA). If that's the operator's setup, swap MD9 for MD2 in the three locations.

12. Fine-Tune Encoder Repurposed for FT8 RX Frequency

Adjusting FT8 audio frequency required programmatic access to ft8RxFreq. No front-panel UI.

What was done

- Repurposed the fine-tune encoder in HOME state: when modulation is FT8_INTERNAL, iFINETUNE_INCREASE / iFINETUNE_DECREASE call ChangeFT8RxFreq(+/- 1) (5 Hz step) instead of AdjustFineTune(+/- 1).
- Both INCREASE and DECREASE branches updated symmetrically.
- Other encoders (volume, filter, main-tune) and the iFINETUNE_* handlers in non-HOME states (UPDATE, CALIBRATE_*) untouched.

Engineering rationale

Fine-tune is the right encoder to repurpose: in FT8 mode, fine-tune of the SSB VFO doesn't apply the same way (FT8 sits at fixed audio offsets within the SSB tune). ChangeFT8RxFreq applies bounds checking (200-2700 Hz) and mirrors to ft8TxFreq when txEqualsRx is true.

End state

On the home screen in FT8 mode: spinning fine-tune adjusts the FT8 audio RX frequency by 5 Hz per click. SSB VFO unchanged. Other modes: encoder retains its original behavior (no regression).

Caveats / deferred work

FT8 pane doesn't display ft8RxFreq yet (status-line pass adds this). 5 Hz step hardcoded. Existing rxList not reclassified on freq change.

13. FT8 Status Line on the Pane

Spinning the encoder gave no visual feedback for ft8RxFreq changes. FT8 state (TX enabled, interval, CQ mode) wasn't visible anywhere.

What was done

- Added DrawFT8StatusLine helper in MainBoard_DisplayFT8.cpp that renders a single-line strip across the top of the FT8 pane.
- Status fields: RX freq, TX freq, TXMODE (ON/OFF), CQ (AUTO/MAN), INT (EVEN/ODD).
- Reduced message-column height to make room (carved off ~20 px).
- Added stale-flag invalidation to all five Change* functions in DSP_FT8.cpp (ChangeFT8RxFreq, ChangeFT8TxFreq, ChangeFT8TxInterval, ChangeFT8CqState, ChangeFT8TxState).

Engineering rationale

Single packed line keeps the status visible without crowding the message columns too much. Each field is fixed-width so visual alignment is consistent. Per-field stale invalidation gives immediate feedback on encoder spins.

End state

Spinning the fine-tune encoder in FT8 mode: RX/TX values update by 5 Hz per click, status line repaints immediately. Future Change* calls (from menu or CAT) get the same instant feedback.

Caveats / deferred work

Status line at default font is ~370 px wide; bigger fonts may clip. Field gaps don't column-align with the message columns (status row is not a header). Target callsign added in a later pass.

14. Slot-Aligned TX Gate (Per-Slot, Even/Odd Interval)

FT8 transmit timing was tied to the decode FSM's TX state, which fires wherever in the slot the cycle happens to land. A real FT8 station needs to start TX at the slot boundary, alternating even (0/30 s) and odd (15/45 s) intervals so two stations in QSO interleave correctly.

What was done

- Added per-tick slot-aligned TX gate at the top of RunFT8DecoderLoop.
- Reads UTC time via now(), computes slot index = t / 15.
- Rising-edge detector (s_lastTxSlotIdx caches across calls) ensures trigger fires at most once per 15 s slot.
- Conditions checked: ft8TxState != 0, no TX in flight, queued message WAITING, slot parity matches ft8IntState (0 = even, 1 = odd).
- Reduced FSM FT8_STATE_TX body to a no-op pass-through (trigger logic moved to the per-tick gate).
- Boot-time safety: txMsgs > 0 guards against spurious TX before any message is queued.

Engineering rationale

Decoupling the trigger from the decode cycle is the right architecture: decode runs once per slot but happens at unpredictable points within the slot, while TX needs slot-boundary precision. Per-tick polling at ~10 ms cadence gives slot-boundary precision within 10 ms.

End state

Operator queues a message, sets ft8TxState=1, picks even or odd interval. Phoenix waits for the next matching slot boundary, fires FT8QueueAndStartTx + PTT_PRESSED. PTT_RELEASED falls automatically when audio drains.

Caveats / deferred work

Sub-second precision (the +500 ms standard offset) added in a later pass. now() resolution is whole seconds; the trigger lands within 10 ms of the boundary.

15. Polyphase Upsampling 12 to 24 kHz

FT8 transmit audio: ft8_lib generates at 12 kHz; Phoenix's TX pipeline expects 24 kHz post-decimation. Initial nearest-neighbor 2x duplication produced spectral images centered at 12 kHz that bled into the audio band when the SSB pipeline ran.

What was done

- Added 32-tap Hamming-windowed sinc 2x interpolator using CMSIS-DSP's arm_fir_interpolate_f32.
- Coefficient buffer (32), state buffer (143 = numTaps/L + blockSize - 1), instance struct -- all file-static globals (~700 bytes SRAM).
- EnsureFT8TxFirInit() lazy initializer: computes coefficients via windowed-sinc with cutoff at fs_out/4 = 6 kHz, normalizes sum to L=2.
- FT8QueueAndStartTx zeroes the FIR delay line at start of each new transmission (prevents bleed across messages).
- Removed unused s_ft8TxSubsample counter from the old design.
- FT8GetNextTxAudioChunk rewritten to use arm_fir_interpolate_f32 in fixed 128-input / 256-output blocks. End-of-message graceful fade-out via zero-padded FIR drain.

Engineering rationale

Nearest-neighbor produces images at f_in_Nyquist that fold into the audio band, producing visible spurs in the RF spectrum. 32-tap Hamming-windowed sinc gives ~50 dB stopband, well below FCC spurious emission limits and the noise floor of typical receivers. Coefficients computed at first-TX time (one-shot, cached).

End state

Same TX flow as before, but the 12 -> 24 kHz step is now polyphase. Spectral images suppressed ~50 dB. ~16-sample group delay (1.3 ms) well within FT8 receiver tolerance.

Caveats / deferred work

Block size hardcoded to 128/256 (matches single existing caller). Filter is simple Hamming; Kaiser/Blackman would push stopband lower at the cost of more taps. Cutoff at input Nyquist (standard placement).

16. BandEQ Bypass During FT8 TX

Phoenix's per-band SSB voice EQ (ED.equalizerXmt[] coefficients) shouldn't shape FT8's GFSK tones — the relative tone amplitudes matter for receiver decode SNR.

What was done

- One-line guard added: BandEQ() inside an if (!ft8tx) block in TransmitProcessing.
- TXGain still runs unconditionally (operator's power setting still applies).

Engineering rationale

BandEQ is voice-specific shaping. For FT8 the eight tones at 6.25 Hz spacing need flat amplitude across the audio passband. SSB pipeline downstream of the BandEQ guard handles modulation correctly with either source.

End state

FT8 audio reaches Hilbert/SSB stage with native amplitude profile intact. SSB voice TX behavior unchanged.

Caveats / deferred work

No per-mode override flag yet. TXGain still applies per-band power calibration tuned for SSB; for absolute FT8 power calibration a separate PowerCal_FT8_* table would be needed.

17. UTC Sub-Second TX Alignment

Slot-aligned TX gate fires at slot-index transition with ~10 ms precision. WSJT-X convention is +500 ms into the slot for clean tone synthesis ramp + relay settling.

What was done

- Added now()-tick subsecond calibration: snapshot millis() at the moment now() advances. Yields ms-within-current-UTC-second with ~10 ms resolution.
- Slot timing model: msInSlot = secInSlot * 1000 + msInSecond.
- Pending/fire/expire state machine: rising edge of slot index sets s_txPendingThisSlot. When msInSlot is in [500, 800) ms AND gating conditions met -> fire. When msInSlot >= 800 ms without firing -> expire silently. Either way the pending flag clears.
- Boot-time safety: first observed second-tick calibrates; before that the gate skips slots.

Engineering rationale

+500 ms gives ~50 ms for relay settling and ~10 ms for GFSK ramp. Receivers tolerate +/- 2.5 s slot timing offset, so [500, 800) ms is comfortably standard. Pending/expire pattern guarantees one-shot per slot.

End state

TX fires at +500-800 ms into a matching slot, on the loop tick that first observes that timing window.

Caveats / deferred work

now() returns whole seconds; calibration relies on observing the tick. ~10 ms loop jitter sets the floor on TX timing precision. Tighter precision would need a hardware-timer-driven trigger.

18. FT8 Menu Page (Variables + Queue Presets)

All FT8 runtime knobs were programmatically accessible but not visible in any menu. No UI to queue TX messages.

What was done

- Bumped primaryMenu[] from 8 to 9 entries; added "FT8" category at the end. MainBoard_Display.h extern updated.
- FT8Options[] array (initially 12 entries):
- Variable controls (variableOption): TX Freq Hz, RX Freq Hz, TX = RX (bool), TX Mode (0/1), Interval (0=ev), CQ Mode (0=man).
- Function entries (functionOption): Queue: CQ / 73 / RR73 / 599 / ID, Cancel TX.
- Preset templates with <CALL> and <GRID> placeholders in DSP_FT8.cpp; ExpandFT8Template() substitutes at queue time so changes to ED.callsign/ED.grid take effect immediately.
- FT8QueueMessageSlot() helper expands template into txBuf[0], marks WAITING.

Engineering rationale

On-radio editing of strings without a USB keyboard is impractical via encoder cycling, so callsign/grid are not on the menu. CAT is the out-of-band path for those. The menu focuses on runtime knobs (settable via standard Phoenix variableOption infrastructure) and one-click queue actions for the standard FT8 message types.

End state

Menu has 6 adjustable variables + 5 one-click queue actions + Cancel TX. Encoder turn adjusts TX/RX freqs in 5 Hz steps with bounds (200-2700 Hz).

Caveats / deferred work

Slot 3 (signal report) puts our own callsign first as a placeholder; real signal-report exchange needs target-callsign tracking (later pass). Preset templates are hardcoded; runtime editing deferred.

19. CL/GR CAT Commands (Callsign/Grid)

Phoenix-specific CAT commands needed so callsign and grid can be set at runtime without recompiling Config.h.

What was done

- Two new CAT commands added: CL (callsign) and GR (grid).
- CL: 13-char payload right-padded with spaces; GR: 6-char payload.
- CL_read returns CL<callsign>; with trimmed callsign. Same for GR_read.
- CL_write / GR_write copy payload, right-trim trailing spaces, write to ED, auto-save via SaveDataToStorage(false).
- rtrim_spaces() helper strips trailing spaces and control bytes.
- Bumped NUM_SUPPORTED_COMMANDS from 27 to 29 (alphabetical insertion: CL between BU and DB; GR between FW and ID).
- Added #include "Storage.h" for SaveDataToStorage.

Engineering rationale

Phoenix's CAT parser uses fixed read/write lengths, so variable-length callsigns are right-padded with spaces by the caller. Auto-save on write makes occasional updates from a CAT tool persist immediately.

End state

CAT clients can now set callsign and grid at runtime. Subsequent FT8 transmissions auto-pick-up the new values via template expansion.

Caveats / deferred work

Strictly fixed-length inputs. SaveDataToStorage blocks ~tens of ms per write. No format validation (real callsigns and Maidenhead grids have specific rules). WSJT-X won't use these (Phoenix-specific).

20. Per-Band FT8 Frequency Presets + Tune Menu Entry

Tuning to the standard FT8 calling frequency on each band required knowing 12 different MHz values. No menu shortcut existed.

What was done

- Added s_ft8BandFreqsHz[NUMBER_OF_BANDS] table in DSP_FT8.cpp with the 12 standard worldwide FT8 dial frequencies (160 m: 1.840 MHz, 80 m: 3.573 MHz, ..., 4 m: 70.100 MHz; 0 = no preset for BAND_GENERAL).
- FT8GetBandFreqHz(band) accessor with range check.
- FT8TuneToBandFreq() looks up the freq for the active VFO's current band and calls Phoenix's existing set_vfo() (declared extern from CAT.cpp).
- Added "Tune to FT8 freq" entry to FT8Options menu (now 13 entries).
- Modulation deliberately not changed — the operator can use DEMODULATION to switch into FT8_INTERNAL separately.

Engineering rationale

set_vfo from CAT.cpp does the same band-aware machinery that the FA CAT command uses (saves last-frequency cache, applies SR offset, dispatches iUPDATE_TUNE). Reusing it keeps the tune behavior identical to a CAT FA write.

End state

From any band, one menu select retunes the active VFO to the standard FT8 calling frequency. Phoenix's lastFrequencies cache makes the round-trip preserve the previous freq.

Caveats / deferred work

Modulation isn't auto-changed (deliberate; combined option added in the next pass). BAND_GENERAL has no preset (silent no-op). 4 m FT8 freq assumes ITU Region 1.

21. One-Click "Go to FT8 + Tune"

Tune to FT8 freq leaves modulation alone (useful for monitoring). A one-click combined action — switch to FT8_INTERNAL AND tune — was the natural quick-start.

What was done

- Added FT8GoToModeAndTune(): mirrors the CAT MD_write pattern (sets both bands[..].mode and ED.modulation = FT8_INTERNAL, fires iMODE), then calls FT8TuneToBandFreq(), then flags PaneSpectrum stale.
- Added "Go to FT8 + tune" menu entry below the existing "Tune to FT8 freq" entry. FT8Options now 14 entries.

Engineering rationale

Single menu select gets the operator fully on the air for FT8: correct modulation, correct frequency, FT8 pane displayed, all FT8 state active.

End state

From any band/mode: select "Go to FT8 + tune". Modulation switches, VFO retunes to band's FT8 calling freq, FT8 pane appears, (FT8) label shown in FreqBandMod pane.

Caveats / deferred work

Doesn't undo prior FT8 settings (queued message could fire on next slot). No auto-revert on band change. Pure tune entry is kept alongside for monitoring use case.

22. USB-Keyboard Text Editor (Soft Modal)

On-radio editing of callsign/grid required a text input mechanism. USB keyboard plumbing exists from earlier; needs a focused-input UI.

What was done

- Architecture: soft modal — no UISm state added (avoids touching StateSmith-generated UISm.cpp/.h). Driven by a global flag that Loop.cpp and DrawDisplay both check.
- New file MainBoard_TextEditor.h (92 lines): public API.
- New file MainBoard_TextEditor.cpp (165 lines): edit-in-snapshot pattern, USB-keyboard char drain in Tick, full-screen render with prompt + edit-box + green caret + footer hint.
- Loop.cpp wire-in: TextEditorTick() called per iteration, HandleButtonPress intercepts MENU_OPTION_SELECT (commit) and HOME_SCREEN (cancel) when active, swallows other buttons.
- MainBoard_Display.cpp wire-in: DrawDisplay checks IsActive at the very top, calls TextEditorRender and returns when active.
- Two new menu entries: "Edit Callsign" / "Edit Grid" with shared FT8MenuSaveAfterCommit callback that calls SaveDataToStorage(false).

Engineering rationale

Soft modal avoids modifying generated UISm code (regenerated from .drawio). Edit-in-snapshot pattern lets cancel cleanly revert. MENU_OPTION_SELECT/HOME_SCREEN buttons mapped to commit/cancel so the operator isn't stuck without a USB keyboard.

End state

Menu select Edit Callsign or Edit Grid: full-screen takeover, type via USB keyboard (printable ASCII + backspace), MENU button commits + auto-saves, HOME button cancels.

Caveats / deferred work

No cursor movement (caret always at end). Single line, max 63 chars. Soft modal — radio state machines (CW keyer 1ms tick, FT8 TX gate) continue to run. Without USB keyboard the editor still activates but can only commit/cancel without char input.

23. Target-Callsign Tracking from RX

Reply preset templates needed to know the QSO partner's callsign. Templates were filling in <CALL> (our own) for the target field, producing syntactically-valid but logically-wrong messages.

What was done

- Added ft8TargetCall[14] global, BSS-initialized to empty.
- AddDecodedMessage auto-update: when a decoded message starts with "CQ " AND target is empty, capture field2 (the CQ caller) as target. Heuristic: only when target is empty so operator-set targets aren't overridden.
- Three target-control functions: FT8SetTargetCall(call) / FT8TargetLatestCQ() (pulls head of cqList) / FT8ClearTarget().
- Added <TARGET> placeholder in ExpandFT8Template (8-char token); falls back to <CALL> when target is empty.
- Updated preset templates to standard FT8 QSO format: Slot 0 unchanged ("CQ <CALL> <GRID>"); slots 1-4 now use "<TARGET> <CALL> ..." prefixes (73, RR73, -10, R-10).
- Status line extended: TGT:<callsign> field added (or TGT:---- when empty).
- Three new menu entries: "Target latest CQ", "Edit Target", "Clear Target" — FT8Options now 19 entries.

Engineering rationale

FT8 QSO sequence (G3PLX 2017 / WSJT-X): operator calls CQ, second station replies with TARGET CALL GRID, exchange continues TARGET CALL <report>/RR73/73. Target tracking is what makes reply templates form real responses. Auto-update from CQ keeps the common case zero-friction.

End state

Receive a CQ -> target auto-set to the CQ caller. Queue any of slots 1-4 -> message correctly addressed to that target. Status line shows current target. Clear/edit/re-target via menu.

Caveats / deferred work

Auto-update only fires on EMPTY target (guards in-progress QSO). Signal report -10 is a placeholder; real reports come from rxBuf[idx].snr (deferred). Target isn't persisted across reboots. No click-on-message-to-target UX (mouse driver exists; binding deferred).

24. USBHost_t36 Vendoring and Repointing

USBHost_t36 was used via the system include <USBHost_t36.h>, picked up from the developer's Teensyduino install. To pin the version Phoenix was developed against, vendoring was needed.

What was done

- Network access from sandbox to GitHub initially blocked; user cloned PaulStoffregen/USBHost_t36.git locally to /Users/jpwatters/src/USBHost_t36.
- Captured commit SHA: 0463fc1f6591c0d303698d7454f6a130cf84dd67 (master HEAD).
- Initially copied to code/src/PhoenixSketch/src/USBHost_t36/ (inside the sketch tree).
- Then moved to code/src/USBHost_t36/ (sibling of PhoenixSketch) per user direction. Examples directory was lost during mv (macOS-mount cross-filesystem semantic) and restored from the original clone.
- VENDORED.md provenance file added: upstream URL, commit SHA, vendor date, license, Phoenix usage, refresh procedure, three options for build-time integration (recommended: symlink to ~/Documents/Arduino/libraries/USBHost_t36).
- Repointed three Phoenix files (FrontPanel_USBHost.cpp, FrontPanel_USBKeyboard.cpp, FrontPanel_USBMouse.cpp) from system <USBHost_t36.h> to the standard angle-bracket form per user request (after a brief detour through the relative-path "../USBHost_t36/..." form), with comments noting the vendored location.

Engineering rationale

Vendoring pins the library version; living at code/src/ as a sibling of PhoenixSketch keeps the sketch tree clean and lets multiple sketches (PhoenixSketch, BoardTest, ATTiny85_On_Off) share the library. The angle-bracket include + symlink-to-Arduino-libraries pattern is the most idiomatic Arduino way to consume a third-party library while keeping the source tree as the single source of truth.

End state

Library at code/src/USBHost_t36/. Phoenix files use #include <USBHost_t36.h>. With the recommended symlink in place (ln -s code/src/USBHost_t36 ~/Documents/Arduino/libraries/USBHost_t36), the Arduino IDE picks it up via the standard library mechanism.

Caveats / deferred work

.git/ remnant (1.7 MB) inside the vendored copy couldn't be removed from the sandbox due to macOS-mount permissions. Examples directory (53 sketches, 1.2 MB) is included. Without the symlink, the IDE will fall back to whatever Teensyduino-bundled USBHost_t36 is installed.

Sources

Source listings for each chapter. Multiple sections per page; individual sections are not split across pages.

1. Architecture Survey: Phoenix vs T-41

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/ (target tree, 87 files baseline)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch.bak/ (untouched backup)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/ (read-only reference fork, never modified)
- /Users/jpwatters/src/PhoenixVxxV02/README.md (Phoenix's own description as a ground-up rewrite)
- /Users/jpwatters/src/T41_Vxx/README.md (T41 fork description by tmr4)

2. NFM Demod Port

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/SDT.h (enum addition)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP.h (declaration)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP.cpp (implementation + dispatcher case)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/Demod.cpp (reference: T41 nfmdemod)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/Demod.h (reference: T41 declarations)

3. CW Message Keyer Port

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_CWMessageKeyer.h (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_CWMessageKeyer.cpp (NEW)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/keyer.cpp (reference, 525 lines)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/keyer.h (reference, 36 lines)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_CWProcessing.h (existing CW receive infrastructure)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/ModeSm.drawio (existing CW transmit FSM)

4. USB Input Bundle Port (Mouse + Keyboard + USB Host)

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/Config.h (flag definition)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/FrontPanel_USBHost.h (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/FrontPanel_USBHost.cpp (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/FrontPanel_USBKeyboard.h (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/FrontPanel_USBKeyboard.cpp (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/FrontPanel_USBMouse.h (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/FrontPanel_USBMouse.cpp (NEW)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/t41USBHost.cpp (reference)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/keyboard.cpp (reference)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/mouse.cpp (reference, action layer NOT ported)
- PJRC USBHost_t36 library (system include initially, vendored later)

5. WSJT-X CAT Comparison and Compatibility Additions (TB, SF)

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/CAT.cpp (extended +94 lines)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/wsjt.cpp (reference; Kenwood TS-890S subset)
- WSJT-X documentation, Kenwood TS-480 manual (referenced for command formatting)

6. PSK31 Primitives Port

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_PSK31.h (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_PSK31.cpp (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP.h (ApproxAtan2 declaration added)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/psk31.cpp (reference, ~995 lines)
- G3PLX 1998 varicode specification

7. FT8 Port: Enum, Dispatcher, Vendored ft8_lib, Phoenix Integration

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/SDT.h (enum addition)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP.h (ft8InternalDemod declaration)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP.cpp (dispatcher case)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.h (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/src/ft8_lib/ (vendored, 24 files, 5,194 LOC)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/ft8.cpp (reference, 1814 lines)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/ft8.h (reference, 113 lines)
- /Users/jpwatters/src/T41_Vxx/T41_SDR/src/ft8_lib/ (upstream KGOBA library, source of vendored copy)
- https://github.com/kgoba/ft8_lib (upstream KGOBA repository)

8. FT8 Message Pane (Three-Column Display)

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_DisplayFT8.h (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_DisplayFT8.cpp (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.h (extern exposure)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp (static removal, stale flag)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_DisplayHome.cpp (dispatch + labels)

9. Loop Wire-In and DEMODULATION Button Cycle Extension

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/Globals.cpp (setup edit + include)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/Loop.cpp (loop edit + 2 cycle handlers + include)

10. FT8-Tuned Receive Filter, Callsign/Grid Persistence, and FT8 Transmit Wiring

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FIR.cpp (filter cases)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/SDT.h (ED fields)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/Storage.cpp (JSON serialize + 2 deserialize sites)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.h (TX API)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp (TX state vars + functions + FSM update)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP.cpp (TransmitProcessing substitution)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/Config.h (MY_CALL default)

11. CAT Recognition for FT8 and NFM Modes

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/CAT.cpp (4 switches updated)

12. Fine-Tune Encoder Repurposed for FT8 RX Frequency

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/Loop.cpp (HOME-state iFINETUNE_INCREASE/DECREASE handlers)

13. FT8 Status Line on the Pane

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_DisplayFT8.cpp (helper + reduced columns)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp (5 Change* functions stale flag)

14. Slot-Aligned TX Gate (Per-Slot, Even/Odd Interval)

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp (gate + FSM trim)

15. Polyphase Upsampling 12 to 24 kHz

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp (FIR machinery + GetNextTxAudioChunk rewrite)
- CMSIS-DSP arm_fir_interpolate_f32 (Teensyduino-bundled)

16. BandEQ Bypass During FT8 TX

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP.cpp (TransmitProcessing BandEQ guard)

17. UTC Sub-Second TX Alignment

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp (subsecond calibration + window logic at top of RunFT8DecoderLoop)

18. FT8 Menu Page (Variables + Queue Presets)

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_Display.h (size bump)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_DisplayMenus.cpp (FT8 page + 6 VariableParameters + wrappers + primaryMenu)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.h (FT8_NUM_TX_PRESETS, queue API)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp (preset templates, ExpandFT8Template, FT8QueueMessageSlot)

19. CL/GR CAT Commands (Callsign/Grid)

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/CAT.cpp (4 handlers + 2 table entries + include)

20. Per-Band FT8 Frequency Presets + Tune Menu Entry

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.h (declarations)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp (table + accessor + tune)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_DisplayMenus.cpp (menu entry)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/CAT.cpp (set_vfo, called via extern)

21. One-Click "Go to FT8 + Tune"

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.h
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_DisplayMenus.cpp

22. USB-Keyboard Text Editor (Soft Modal)

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_TextEditor.h (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_TextEditor.cpp (NEW)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/Loop.cpp (Tick + button intercept)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_Display.cpp (DrawDisplay intercept)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_DisplayMenus.cpp (2 menu entries + wrappers)

23. Target-Callsign Tracking from RX

- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.h (target API)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/DSP_FT8.cpp (global + auto-update + 3 functions + <TARGET> in ExpandFT8Template + new templates)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_DisplayFT8.cpp (status line TGT field)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/MainBoard_DisplayMenus.cpp (3 menu entries + wrappers)

24. USBHost_t36 Vendoring and Repointing

- /Users/jpwatters/src/PhoenixVxxV02/code/src/USBHost_t36/ (vendored, 85 files)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/USBHost_t36/VENDORED.md (provenance)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/FrontPanel_USBHost.cpp (include line)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/FrontPanel_USBKeyboard.cpp (include line)
- /Users/jpwatters/src/PhoenixVxxV02/code/src/PhoenixSketch/FrontPanel_USBMouse.cpp (include line)
- /Users/jpwatters/src/USBHost_t36/ (original local clone)
- https://github.com/PaulStoffregen/USBHost_t36 (upstream repo)
- Commit SHA: 0463fc1f6591c0d303698d7454f6a130cf84dd67

Addendum: T-41 Menu Hierarchy

Reference reproduction of T-41's menu structure as defined in T41_SDR/Menu.cpp. Useful when comparing against Phoenix's primaryMenu[] (which is FT8-aware but otherwise has parallel categories: RF Options, CW Options, Audio Options, Display, EEPROM, Calibration, Diagnostics, Microphone, FT8). Phoenix combines T-41's separate "Spectrum Options" into Display, has no Bearing or Beacon Monitor categories, and adds an explicit FT8 category.

From T41_SDR/Menu.cpp::topMenus[] (14 primary categories, TOP_MENU_COUNT = 14). Secondary entries from T41_SDR/Menu.cpp::secondaryChoices[][8]. RF Options and Calibrate vary by hardware variant (V12, V11, vPS, vPT, vMP, vAP, minHardware) -- the entries shown are for the V12 build matching Phoenix's target hardware (T41_SDR/hardware/v12/hardware.h).

Primary categories

T-41's primary menu (TOP_MENU_COUNT = 14). The trailing "Cancel" entry is the menu's exit option, not a separate function.

Primary	Secondary entries
CW Options	WPM, Key Type, CW Filter, Paddle Flip, Sidetone Vol, Xmit Delay, Cancel
RF Options	Power level, Gain, Atten In, Atten Out, Cancel
VFO Select	VFO A, VFO B, Split, Cancel
EEPROM	Save Current, Set Defaults, Get Favorite, Set Favorite, EEPROM-->SD, SD-->EEPROM, SD Dump, Cancel
AGC	Off, Long, Slow, Medium, Fast, Cancel
Spectrum Options	20 dB/unit, 10 dB/unit, 5 dB/unit, 2 dB/unit, 1 dB/unit, Cancel
Mic Gain	Set Mic Gain, Cancel
Mic Comp	On, Off, Set Threshold, Cancel
EQ Rec Set	On, Off, EQSet, Cancel
EQ Xmt Set	On, Off, EQSet, Cancel
Calibrate	Freq Cal, Rec Cal, Xmit Cal, Two Tone, CW PA Cal, SSB PA Cal, Cancel
Bearing	Show Map, Set Prefix, Cancel
Beacon Monitor	On, Off, Cancel
Cancel	(no submenu)

Notes on hardware variation

Two primary categories vary by hardware build: RF Options and Calibrate. T-41 conditionally includes them via macros defined in T41_SDR/hardware/<variant>/hardware.h. Phoenix targets the V12 hardware, so the V12 entries are listed above. For reference, the other variants differ as follows:

- V12: RF -> Power level, Gain, Atten In, Atten Out, Cancel | Cal -> Freq Cal, Rec Cal, Xmit Cal, Two Tone, CW PA Cal, SSB PA Cal, Cancel
- V11: RF -> Power level, Gain, Atten In, Atten Out, Cancel | Cal -> (varies; see hardware/v11/hardware.h)
- vPS: RF -> Power level, Gain, Cancel | Cal -> Frequency, CW Pwr, SSB PA, IQ, Two Tone, Cancel
- vPT: RF -> Power level, Gain, Cancel | Cal -> (varies)
- vMP, vAP, minHardware: smaller subsets per hardware/<variant>/hardware.h

Phoenix vs T-41 menu mapping

Where T-41 categories map onto Phoenix's primaryMenu[9]. "--" means Phoenix has no equivalent; "+" means Phoenix has a category T-41 lacks.

T-41 primary	Phoenix equivalent
CW Options	CW Options
RF Options	RF Options
VFO Select	(handled via VFO_TOGGLE button + MAIN_TUNE_INCREMENT, no menu)
EEPROM	EEPROM
AGC	(part of RF / Audio Options)
Spectrum Options	Display
Mic Gain	Microphone
Mic Comp	(--)
EQ Rec Set	(part of Display / accessed via DFE button)
EQ Xmt Set	(part of Display / accessed via DFE button)
Calibrate	Calibration
Bearing	(--)
Beacon Monitor	(--)
(--)	Audio Options (+)
(--)	Diagnostics (+)
(--)	FT8 (+; added by this work)